

Kemika Sherchan
PRG 400
Week 1 Assignment

Object-Oriented Programming(OOP) is a type of programming where we structure the code in terms of classes and objects. It assists in making easy management, reusability and organization of programs. Principles of OOP that have been illustrated:
Encapsulation Protecting data within a class.
Inheritance -One class utilizing properties of another class.
Polymorphism -Same method acting differently with different objects.

Encapsulation

Encapsulation means hiding internal data and allowing access only through methods. It protects data from direct modification outside the class.

Example: A class hides private variables using __.

```
In [1]: # Encapsulation

class BankAccount:

    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance    # private variable (encapsulated)

    # Getter method to access balance safely
    def get_balance(self):
        return self.__balance

    # Method to deposit money
    def deposit(self, amount):
        self.__balance += amount
        print("Deposit successful")

    # Method to withdraw money
    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
            print("Withdrawal successful")
        else:
            print("Insufficient balance")

# Creating object
```

```
account = BankAccount("Kemika", 1000)

# Accessing through methods
account.deposit(500)
account.withdraw(200)

# Accessing private data using getter
print("Current Balance:", account.get_balance())

# Direct access would fail
# print(account.__balance) # This will cause an error
```

Deposit successful
Withdrawal successful
Current Balance: 1300

Code Explanation

class BankAccount: This is a class that builds a bank account.

The constructor executable when an object is created is called `__init__()`

self is a private variable: it begins with `__`.

One cannot access the private variables directly outside the class.

The alteration of the balance is done safely through methods such as deposit and withdraw.

get-balance can be used to read the balance.

Program Flow

BankAccount is defined.

Object account is opened with the owner Kemika and balance 1000.

The program will save 500 into the account.

Then it withdraws 200.

Lastly, the balance left is printed by the program.

Inheritance

The inheritance enables a child class to inherit properties and methods of a parent class. It assists in minimizing repetition of codes and establishes connections among classes.
Example: Animal. h2: Dog and Cat.

```
In [3]: # Inheritance

# Parent class
class Animal:

    def __init__(self, name):
        self.name = name

    def speak(self):
        print("Animal makes a sound")

# Child class inheriting from Animal
```

```
class Dog(Animal):  
    def speak(self):  
        print(self.name, "barks")  
  
# Another child class  
class Cat(Animal):  
    def speak(self):  
        print(self.name, "meows")  
  
# Creating objects  
dog1 = Dog("Buddy")  
cat1 = Cat("Whiskers")  
  
dog1.speak()  
cat1.speak()
```

Buddy barks

Whiskers meows

Code Explanation

Animal is the parent class.

It has an object-creator and a speak method.

Animal has child classes of Dog and Cat.

They share the name with parent class property.

They are selecting the speak method and defining their behavior.

Program Flow

The parent class Animal is generated.

It has two child classes Dog and Cat.

Objects dog1 and cat1 are generated.

Every object invokes a version of the speak method.

The program displays the sounds of every animal.

Polymorphism

Polymorphism implies that the same method name can be used to do various things in a specific situation determined by the object that invokes it. This permits various classes to adopt the same approach to suit themselves.

Example: dog and cat do not behave identically with speak.

In [6]: *# Polymorphism*

```
class Animal:  
    def speak(self):  
        pass
```

```
class Dog(Animal):
    def speak(self):
        return "Dog barks"

class Cat(Animal):
    def speak(self):
        return "Cat meows"

# Function demonstrating polymorphism
def animal_sound(animal):
    print(animal.speak())

# Creating objects
dog = Dog()
cat = Cat()

# Same function works for different objects
animal_sound(dog)
animal_sound(cat)
```

Dog barks
Cat meows

Code Explanation

A method speak is defined in the Animal class.

Dog class replaces speak() to give Dog barks.

The Cat class rewrites the speak method in order to give Cat meows in response.

The animal sound function takes any object and calls speak.

Although they are utilizing the same object, various outputs are visible based on the object.

Program Flow

Animal, Dog, and Cat classes are delineated.

Dog and cat objects are made.

Animal sound is a function that is invoked with various objects.

Every object has its own implementation of the speak method.

This final example combines inheritance and polymorphism.

```
In [7]: animals = [Dog(), Cat()]

for animal in animals:
    print(animal.speak())
```

Dog barks
Cat meows

Conclusion

Object-Oriented Programming (OOP) assists the developer to develop software by assembling data and methods on the data within the classes. This simplifies the process of organizing, understanding and reusing programs. OOP allows them to design their code in a more understandable and manageable form to the programmer as well as the computer programs.

Encapsulation

The key data within a class is safeguarded with the help of encapsulation. The class does not have other functions of the program changing the data directly, but instead allows direct access or modification of the data through certain methods provided. In our case, the balance in the bank account was a secret variable with two underscores before it (`\_\_balance`). It implies that the balance cannot be altered immediately out of the classroom. We instead update the balance through such methods as deposition and withdrawal which are safe. This would enhance safety of the program and ensure that valuable data is not changed accidentally.

Inheritance

The concept of Inheritance enables a class to reuse other properties and methods of another class. This assists in lowering the redundancy of the code and it establishes a very clear connection between classes. In our scenario we have a parent class named as animal which has a shared property (name) and a shared method (speak). We then developed child classes like Dog and Cat which are children of the Animal class. These child classes inherit the properties of the parent class without making them to lose any behaviors of their own. This ensures that the program is more efficient and manageable.

Polymorphism

Polymorphism lets method names act differently based on the object that invokes them. In our example, the Dog and Cat classes have a method in them named as speak. Nonetheless, when the method is called, every class gives a different result. The Dog object writes out Dog barks whereas the Cat object writes out Cat meows. Although the name of the method does not differ, the behavior varies with the object. This gives the programs more flexibility and easier to grow in the future.

Therefore, these three concepts of OOP assist developers in developing programs in a superior manner. With encapsulation, inheritance, and polymorphism, the programmers will be able to develop the software in a more structured, manageable, and flexible way as the program expands.